



Hands-On Lab Exercise 11: Monitoring Automation Using Shell Script

Topic: System Monitoring and Health Check Automation

Author: Dr. Sandeep Kumar Sharma

Monitoring is one of the most important responsibilities in system administration, DevOps, and cloud operations. Every system must be continuously observed to ensure that it is working properly. Servers can run out of disk space, memory can become full, CPU usage can increase, services can stop, and systems can become unstable. Without monitoring, problems are discovered only after failures happen.

Shell scripting helps in building simple but powerful monitoring systems. Using shell scripts, we can automatically check system health, collect system information, detect problems, and generate reports. This transforms manual checking into automated monitoring. Scripts can be scheduled using cron jobs to run automatically at regular intervals, making monitoring continuous and reliable.

Monitoring automation means the system checks itself, instead of waiting for a human to check it. This is the foundation of professional IT operations, DevOps, cloud management, and infrastructure reliability.

In simple words, monitoring automation is the process of using scripts to continuously watch the system and detect problems before they become failures.



Learning Objective

- Understand monitoring automation
 - Learn how scripts check system health
 - Learn how to collect system information
 - Learn how to automate health checks
 - Learn how to build monitoring scripts
-



Learning Outcome

After this lab, participants will: - Build system monitoring scripts - Automate health checks - Monitor system resources - Create basic monitoring tools - Understand real-world monitoring automation



What Should Be Monitored?

Systems are monitored for: - CPU usage - Memory usage - Disk usage - System uptime - Running services - System load - User activity - System performance



Example 1: Basic System Health Monitor

Step 1: Create Script

```
touch monitor1.sh
```

Step 2: Open File

```
nano monitor1.sh
```

Step 3: Write Script

```
#!/bin/bash

echo "System Health Report"
echo "-----"

echo "Date and Time:"
date

echo "-----"

echo "System Uptime:"
uptime

echo "-----"

echo "CPU Load:"
cat /proc/loadavg

echo "-----"
```

```
echo "Memory Usage:"
free -h

echo "-----"

echo "Disk Usage:"
df -h
```

Step 4: Permission

```
chmod +x monitor1.sh
```

Step 5: Run

```
./monitor1.sh
```



Example 2: Disk Space Monitoring Script



Purpose:

Check disk usage and warn if disk is high

```
#!/bin/bash

usage=$(df / | tail -1 | awk '{print $5}' | sed 's/%//')

echo "Disk Usage: $usage%"

if [ $usage -gt 80 ]
then
    echo "Warning: Disk usage is above 80%"
else
    echo "Disk usage is normal"
fi
```



Example 3: Memory Monitoring Script

```
#!/bin/bash

free_mem=$(free -m | awk 'NR==2{print $4}')

echo "Free Memory: $free_mem MB"

if [ $free_mem -lt 500 ]
then
    echo "Warning: Low memory available"
else
    echo "Memory level is healthy"
fi
```



Example 4: Service Monitoring Script



Purpose:

Check if a service is running

```
#!/bin/bash

service_name=ssh

status=$(systemctl is-active $service_name)

echo "Service: $service_name"

echo "Status: $status"

if [ "$status" = "active" ]
then
    echo "Service is running properly"
else
    echo "Service is NOT running"
    echo "Restarting service..."
    systemctl restart $service_name
    echo "Service restarted"
fi
```



Example 5: Full Monitoring Report Script

```
#!/bin/bash

report_file="system_report.txt"

echo "System Monitoring Report" > $report_file
echo "-----" >> $report_file

date >> $report_file

echo "Uptime:" >> $report_file
uptime >> $report_file

echo "CPU Load:" >> $report_file
cat /proc/loadavg >> $report_file

echo "Memory Usage:" >> $report_file
free -h >> $report_file

echo "Disk Usage:" >> $report_file
df -h >> $report_file

echo "Report generated in system_report.txt"
```

Understanding the Flow

Scripts collect system data, analyze it, apply conditions, and generate outputs. This is the basic structure of monitoring systems. Advanced monitoring tools follow the same logic, but at a larger scale.

Key Understanding

Monitoring automation transforms systems into self-observing systems. Scripts become guardians of system health.

Summary

- Monitoring is essential for reliability
 - Automation prevents failures
 - Scripts enable continuous monitoring
 - Health checks are foundation of DevOps
 - Monitoring is proactive, not reactive
-

End of Lab

This lab introduces **automated system monitoring**.

Now scripts are not just automation tools — They are **system guardians**.

This is core to: - DevOps operations - Cloud reliability - Site reliability engineering - Infrastructure automation - Production systems

Author:

Dr. Sandeep Kumar Sharma

LinkedIn:

linkedin.com/in/sandeep-kaushik-2a345856

Medium Blog:

medium.com/@shyamsandeep28